

Disciplina: Aprendizado Profundo

Aula 5: Otimização para Aprendizado Profundo

Eliezer de Souza da Silva
sereliezer.github.io
eliezer.silva@ufc.br

Mestrado e Doutorado em Ciência da Computação, Universidade Federal do
Ceará (MDCC / UFC)

29 de Setembro de 2025

Roteiro da Aula de Hoje

- 1 Revisão e Motivação
- 2 Fundamentos de Otimização
- 3 Otimizadores Avançados
- 4 Fronteiras da Pesquisa em Otimização de Redes Neurais
- 5 Próximos Passos

Recapitulando as Aulas Anteriores

Onde Paramos

- Construímos o MLP, um aproximador universal de funções f_θ .
- Derivamos o Backpropagation, um algoritmo para calcular os gradientes da função de custo / loss $\nabla_\theta J$.
- Implementamos um MLP do zero com NumPy, usando a Descida de Gradiente Estocástica (SGD) para atualizar os pesos: $\theta \leftarrow \theta - \eta \nabla_\theta J$.

A Pergunta Central de Hoje

A descida de gradiente é a única forma de otimizar? Existem métodos melhores? Como a geometria da função de custo afeta o treinamento?

Notação e Elementos Matemáticos I

Parâmetros e a Função de Custo

- **Vetor de Parâmetros (θ):** Para a análise, vamos juntar todos os pesos e vieses da rede, $\{W^{(l)}, b^{(l)}\}$, em um único vetor de parâmetros de alta dimensão, $\theta \in \mathbb{R}^P$. Portanto a nossa rede é uma função paramétrica f_θ .
- **Função de Custo ($J(\theta)$):** É uma função escalar ($J : \mathbb{R}^P \rightarrow \mathbb{R}$) que mapeia o vetor de parâmetros para uma medida de erro. Nós a visualizamos como uma “paisagem de custo” sobre o espaço de parâmetros. Quando necessário vamos deixar explícita a dependência dos dados.

Notação e Elementos Matemáticos II

Derivadas de Primeira Ordem: O Gradiente

O **gradiente** de J em relação a θ é o vetor de todas as derivadas parciais de primeira ordem:

$$\nabla J(\theta) = \left[\frac{\partial J}{\partial \theta_1}, \frac{\partial J}{\partial \theta_2}, \dots, \frac{\partial J}{\partial \theta_P} \right]^T$$

Notação e Elementos Matemáticos III

Derivadas de Segunda Ordem: A Hessiana

A **matriz Hessiana** $H_J(\theta)$ de J é a matriz de todas as derivadas parciais de segunda ordem:

$$H_J(\theta)_{ij} = (\nabla^2 J(\theta))_{ij} = \frac{\partial^2 J}{\partial \theta_i \partial \theta_j}$$

Ela descreve a **curvatura local** da paisagem de custo, sendo fundamental para métodos de otimização de segunda ordem (como o método de Newton).

Notação e Elementos Matemáticos IV

Nota: Gradiente vs. Jacobiana

O gradiente é um caso especial da Jacobiana. Para uma função vetorial $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, a Jacobiana é a matriz $m \times n$ de todas as derivadas parciais. Se a função é escalar ($m = 1$), a Jacobiana é um vetor linha, e seu transposto é o gradiente.

Qual é a Direção de Maior Descida? I

Aproximação Local da Função de Custo

Para encontrar a melhor direção para “descer” na paisagem de custo a partir de um ponto θ , podemos usar uma aproximação linear da função J através da sua expansão de Taylor de primeira ordem. Para um passo pequeno η em uma direção unitária v (onde $\|v\| = 1$):

$$J(\theta + \eta v) \approx J(\theta) + \eta \nabla J(\theta)^T v$$

A mudança no custo é, portanto, $\Delta J \approx \eta \nabla J(\theta)^T v$.

Qual é a Direção de Maior Descida? II

O Problema de Otimização

Queremos encontrar a direção v que **minimiza** essa mudança, ou seja, que torna o produto escalar $\nabla J(\theta)^T v$ o mais negativo possível, sujeito à restrição de que v seja um vetor unitário.

$$v^* = \arg \min_{v \text{ s.t. } \|v\|=1} \nabla J(\theta)^T v$$

Qual é a Direção de Maior Descida? III

A Solução

Lembrando da definição do produto escalar:

$$\mathbf{a}^T \mathbf{b} = \|\mathbf{a}\| \cdot \|\mathbf{b}\| \cos(\phi).$$

Como $\|\mathbf{v}\| = 1$, temos:

$$\nabla J(\theta)^T \mathbf{v} = \|\nabla J(\theta)\| \cos(\phi)$$

Este valor é minimizado quando $\cos(\phi) = -1$, o que ocorre quando $\mathbf{v}$ aponta na direção exatamente oposta ao gradiente $\nabla J(\theta)$.

Portanto, a direção de maior descida é:

$$\mathbf{v}^* = -\frac{\nabla J(\theta)}{\|\nabla J(\theta)\|}$$

Por que a Descida de Gradiente é o Padrão em Deep Learning? I

Simplicidade e Eficácia

Apesar de existirem métodos de otimização mais complexos, a família de algoritmos baseada na descida de gradiente (GD, SGD, Adam) domina o campo por duas razões principais:

Por que a Descida de Gradiente é o Padrão em Deep Learning? II

1. Custo Computacional

- **Descida de Gradiente (1ª Ordem):** Requer apenas o cálculo do gradiente (∇J), que o Backpropagation faz de forma muito eficiente. O custo por iteração é baixo.
- **Método de Newton (2ª Ordem):** Requer o cálculo do gradiente e da **matriz Hessiana** ($\nabla^2 J$), além da *inversão* desta matriz. Para uma rede com milhões de parâmetros, isso é computacionalmente intratável.

Por que a Descida de Gradiente é o Padrão em Deep Learning? III

2. Escalabilidade com Dados

- A descida de gradiente funciona perfeitamente com a aproximação estocástica (Mini-Batch SGD), permitindo treinar modelos com bilhões de parâmetros em datasets massivos.
- O “ruído” introduzido pelo SGD ajuda o otimizador a navegar a paisagem de custo não-convexa das redes neurais, evitando mínimos locais ruins e pontos de sela.

O Cenário Ideal: Otimização Convexa

Por que Estudar Convexidade?

Embora a função de custo de uma rede neural seja não-convexa, o estudo da otimização convexa é fundamental por duas razões:

- 1 É a teoria que entendemos bem:** A otimização convexa nos fornece um conjunto de ferramentas e garantias teóricas robustas sobre a convergência dos algoritmos.
- 2 Aproximação Local:** Muitas funções não-convexas se comportam localmente como funções convexas, então a intuição e os métodos do mundo convexo servem como base para a otimização em geral.

O Cenário Ideal: Otimização Convexa I

Definições Fundamentais

- **Conjunto Convexo:** Um conjunto $\mathcal{C}$ é convexo se, para quaisquer $x, y \in \mathcal{C}$, o segmento de linha entre eles está contido em $\mathcal{C}$.
- **Função Convexa:** Uma função J é convexa se seu domínio é um conjunto convexo e, para quaisquer x, y no domínio e $\lambda \in [0, 1]$:

$$J(\lambda x + (1 - \lambda)y) \leq \lambda J(x) + (1 - \lambda)J(y)$$

O Cenário Ideal: Otimização Convexa II

A Propriedade de Ouro: Mínimo Local é Global

Para uma função convexa, **qualquer mínimo local é também um mínimo global**. Isso transforma a busca pelo ótimo em um problema muito mais tratável.

O Cenário Ideal: Otimização Convexa III

Prova Intuitiva (por Contradição)

- 1 **Assuma o oposto:** Suponha que θ^* é um mínimo local numa vizinhança $\|\theta^* - \theta\|_2 \leq r$, mas *não* é um mínimo global. Isso significa que existe algum outro ponto θ_g fora dessa vizinhança tal que $J(\theta_g) < J(\theta^*)$. Considere um ponto $\theta_{\text{novo}} = \lambda\theta_g + (1 - \lambda)\theta^*$ na linha entre θ^* e θ_g para qualquer $\lambda \in (0, 1)$:

$$J(\theta_{\text{novo}}) \leq \lambda J(\theta_g) + (1 - \lambda)J(\theta^*)$$

- 2 **Contradição:** Como $J(\theta_g) < J(\theta^*)$, temos a desigualdade estrita:

$$J(\theta_{\text{novo}}) < \lambda J(\theta^*) + (1 - \lambda)J(\theta^*) = J(\theta^*)$$

Encontramos um ponto θ_{novo} , arbitrariamente próximo de θ^* , com um valor de custo menor. Isso **contradiz** a suposição de que θ^* era um mínimo local.

Analizando a Geometria: Suavidade e Convexidade Forte I

L -suavidade (Gradiente L -Lipschitz)

Uma função J é L -suave se seu gradiente não muda abruptamente. Matematicamente, existe $L > 0$ tal que:

$$\|\nabla J(\theta_1) - \nabla J(\theta_2)\|_2 \leq L\|\theta_1 - \theta_2\|_2$$

Intuição: A taxa de crescimento da função é limitada superiormente por uma parábola. Garante que não “pulemos” o mínimo.

Analizando a Geometria: Suavidade e Convexidade Forte II

μ -convexidade forte (Strong Convexity)

Uma função J é μ -fortemente convexa se ela é “suficientemente curva”. Matematicamente, existe $\mu > 0$ tal que a função é limitada inferiormente por uma parábola:

$$J(\theta_2) \geq J(\theta_1) + \nabla J(\theta_1)^T (\theta_2 - \theta_1) + \frac{\mu}{2} \|\theta_2 - \theta_1\|_2^2$$

Intuição: Garante a existência de um único mínimo global e evita regiões planas.

Analizando a Geometria: Suavidade e Convexidade Forte III

Relações entre as Propriedades

Estas propriedades não são equivalentes e descrevem diferentes aspectos da geometria da função.

- **Convexidade Forte $\implies$ Convexidade:** Toda função fortemente convexa é também convexa. O contrário não é verdadeiro (e.g., $J(\theta) = \theta$).
- **L-suavidade e Convexidade são independentes:**
 - $J(\theta) = |\theta|$ é convexa, mas não é suave (não diferenciável em $\theta = 0$).
 - $J(\theta) = -\theta^2$ é L-suave, mas não é convexa (é côncava).
- Para convergência rápida, o cenário ideal é ter uma função que seja **L-suave** e **μ -fortemente convexa**.

Provando o Lema da Descida I

Objetivo

Queremos provar que a L-suavidade implica no limite superior quadrático. Começamos com a L-suavidade:

$$\|\nabla J(\theta_1) - \nabla J(\theta_2)\|_2 \leq L\|\theta_1 - \theta_2\|_2$$

E queremos chegar em:

$$J(\theta_2) \leq J(\theta_1) + \nabla J(\theta_1)^T(\theta_2 - \theta_1) + \frac{L}{2}\|\theta_2 - \theta_1\|_2^2$$

Provando o Lema da Descida II

- 1 Pelo Teorema Fundamental do Cálculo**, podemos expressar a mudança na função como a integral de linha $\int \nabla J(\phi(t))^T D[\phi] dt$ do seu gradiente:

$$J(\theta_2) - J(\theta_1) = \int_0^1 \nabla J(\theta_1 + t(\theta_2 - \theta_1))^T (\theta_2 - \theta_1) dt$$

- 2 Subtraímos** $\nabla J(\theta_1)^T (\theta_2 - \theta_1)$ de ambos os lados:

$$J(\theta_2) - J(\theta_1) - \nabla J(\theta_1)^T (\theta_2 - \theta_1) = \int_0^1 [\nabla J(\theta_{1,t}) - \nabla J(\theta_1)]^T (\theta_2 - \theta_1) dt$$

onde $\theta_{1,t} = \theta_1 + t(\theta_2 - \theta_1)$.

Provando o Lema da Descida III

3 Aplicamos a desigualdade de Cauchy-Schwarz e a L-suavidade:

$$\begin{aligned}\text{Lado Direito} &\leq \int_0^1 \|\nabla J(\theta_{1,t}) - \nabla J(\theta_1)\|_2 \cdot \|\theta_2 - \theta_1\|_2 dt \\ &\leq \int_0^1 L \|\theta_{1,t} - \theta_1\|_2 \cdot \|\theta_2 - \theta_1\|_2 dt \\ &= \int_0^1 L \cdot t \|\theta_2 - \theta_1\|_2 \cdot \|\theta_2 - \theta_1\|_2 dt \\ &= L \|\theta_2 - \theta_1\|_2^2 \int_0^1 t dt = \frac{L}{2} \|\theta_2 - \theta_1\|_2^2\end{aligned}$$

4 **Reorganizando**, obtemos o Lema da Descida, como queríamos demonstrar.

Taxas de Convergência da Descida de Gradiente I

O Primeiro Passo da Prova de Convergência

A partir da definição de L-suavidade, podemos provar que a cada passo da descida de gradiente ($\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$), o valor da função diminui:

$$J(\theta_{t+1}) \leq J(\theta_t) - \eta \left(1 - \frac{L\eta}{2}\right) \|\nabla J(\theta_t)\|_2^2$$

Isso garante a convergência para um ponto estacionário, desde que a taxa de aprendizado η seja escolhida corretamente (exercício extra da lista).

Conclusão: Convexidade e Convergência Descida de Gradiente I

Resumo das Taxas de Convergência

- Para funções L-suaves e **convexas**, a descida de gradiente converge com uma taxa de $O(1/t)$. Para chegar a uma precisão ϵ , precisamos de $O(1/\epsilon)$ iterações.
- Para funções L-suaves e μ -**fortemente convexas**, a descida de gradiente converge com uma taxa **linear** (geométrica) de $O((1 - \mu/L)^t)$. Para chegar a uma precisão ϵ , precisamos de $O(\log(1/\epsilon))$ iterações, o que é exponencialmente mais rápido!

Conclusão: Convexidade e Convergência Descida de Gradiente II

Lista de Exercícios

A derivação detalhada de algumas dessas taxas de convergência é um dos exercícios da nossa próxima lista de problemas.

O Desafio: Otimização Não-Convexa em Deep Learning I

A Paisagem de Custo das Redes Neurais

A função de custo $J(\theta)$ de uma rede neural é **altamente não-convexa**.

- **Múltiplos Mínimos Locais:** Existem muitos pontos onde $\nabla J(\theta) = 0$.
- **Pontos de Sela (Saddle Points):** Pontos onde o gradiente é zero, mas não são mínimos. São um grande obstáculo para a otimização.
- **Platôs:** Regiões vastas e quase planas onde o gradiente é muito pequeno, tornando o progresso lento.

O Desafio: Otimização Não-Convexa em Deep Learning II

O que o SGD Realmente Encontra?

Não buscamos o mínimo global (que é computacionalmente intratável). O objetivo é encontrar um mínimo local “bom” o suficiente para generalizar bem. O ruído do SGD é benéfico aqui, pois ajuda o otimizador a “escapar” de mínimos locais ruins e pontos de sela. Lembrando que o objetivo da minimização do erro empírico regularizado implica que procuramos regiões com certas propriedades, não apenas o mínimo global.

Além do SGD: Acelerando a Convergência

A Limitação do SGD Padrão

O SGD padrão tem uma única taxa de aprendizado (η) para todos os parâmetros e não tem “memória” dos passos anteriores. Isso pode levar a dois problemas:

- **Oscilações:** Em vales estreitos e alongados, o gradiente pode oscilar entre as “paredes” do vale, levando a uma convergência lenta.
- **Taxa de Aprendizado Única:** Diferentes parâmetros podem precisar de diferentes tamanhos de passo (e.g., pesos de bias vs. pesos de features).

Lidando com Termos Não-Suaves: Métodos Proximais I

O Problema: Funções de Custo Não-Diferenciáveis

E se a nossa função de custo tiver uma parte não-suave (não-diferenciável), como na regularização L1 ou ReLUs, $J(\theta) = \hat{R}(\theta) + \lambda \|\theta\|_1$? A descida de gradiente padrão não se aplica.

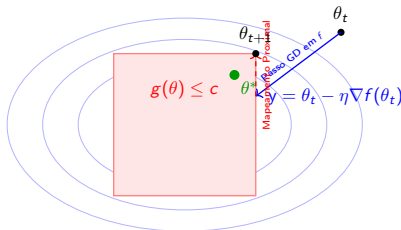


Figura: Visualização do passo proximal.

Lidando com Termos Não-Suaves: Métodos Proximais II

A Ideia: Separar e Mapear

A descida de gradiente proximal separa o problema em duas partes:

- 1 Dar um passo de descida de gradiente normal na parte **suave** da função ($f(\theta) = \hat{R}(\theta)$).
- 2 Aplicar um “operador proximal” que lida com a parte **não-suave** ($g(\theta) = \lambda ||\theta||_1$).

Lidando com Termos Não-Suaves: Métodos Proximais III

Atualização Proximal

A regra de atualização é:

$$\theta_{t+1} \leftarrow \text{prox}_{\eta g}(\theta_t - \eta \nabla f(\theta_t))$$

O operador proximal, $\text{prox}_g(y)$, encontra um ponto que é próximo de y mas que também minimiza g . Para a regularização L1 temos **limiarização suave (soft-thresholding)**, que zera os pesos pequenos.

Lidando com Termos Não-Suaves: Métodos Proximais IV

Relevância para Deep Learning

Embora não sejam usados diretamente na forma pura para treinar a maioria das redes, os conceitos de pré-condicionamento (adaptar a atualização por parâmetro) e lidar com não-suavidades (como a da ReLU) são ideias centrais que inspiram os otimizadores modernos.

Corrigindo a Geometria: Pré-condicionamento I

O Problema de Mal Condicionamento

Em vales longos e estreitos (problemas mal condicionados), o gradiente $-\nabla J(\theta)$ não aponta para o mínimo, causando oscilações e convergência lenta.

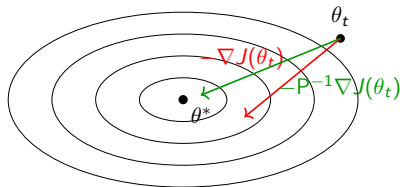


Figura: O gradiente padrão (vermelho) oscila, enquanto o gradiente pré-condicionado (verde) aponta para o ótimo.

Corrigindo a Geometria: Pré-condicionamento II

A Ideia: Reescalar o Espaço

A descida de gradiente pré-condicionada transforma o espaço de parâmetros para que as curvas de nível se tornem mais “circulares” e uniformes, fazendo com que o gradiente aponte diretamente para o mínimo. A regra de atualização se torna:

$$\theta_{t+1} \leftarrow \theta_t - \eta P^{-1} \nabla J(\theta_t)$$

Onde P é a matriz de pré-condicionamento. Uma escolha ideal (mas impraticável) para P é a matriz Hessiana, o que nos leva ao método de Newton.

A Ponte: De Pré-condicionamento a Momento I

O Desafio: A Matriz Hessiana é Intratável

O pré-condicionador ideal é a Hessiana, $P = \nabla^2 J(\theta)$, que captura a curvatura da função de custo. No entanto, para uma rede com milhões de parâmetros, calcular e inverter esta matriz é impossível.

A Ponte: De Pré-condicionamento a Momento II

Uma Ideia: Aproximação de Baixo Posto (Low-Rank)

E se, em vez de usar a Hessiana completa, a aproximássemos usando apenas suas direções de maior curvatura?

- Poderíamos tentar encontrar os principais autovetores da Hessiana (que correspondem a essas direções) usando métodos iterativos como o método da potência.
- O problema é que mesmo isso requer produtos matriz-vetor com a Hessiana, o que ainda é computacionalmente caro.

A Ponte: De Pré-condicionamento a Momento III

O Momento como uma Aproximação de Posto 1

É aqui que surge a conexão com o método do momento.

- Em um vale estreito, os gradientes apontarão consistentemente ao longo do eixo do vale.
- O buffer de momento, v_t , ao acumular uma média móvel dos gradientes passados ($v_t \approx \sum_{i < t} \nabla J_i$), essencialmente “aprende” esta direção dominante.
- A atualização de pesos, $\theta_{t+1} \leftarrow \theta_t - v_{t+1}$, pode ser vista como um passo de descida de gradiente que foi pré-condicionado por uma aproximação de **posto 1 (rank-1)** da Hessiana, construída a partir da história recente dos gradientes.

A Ponte: De Pré-condicionamento a Momento IV

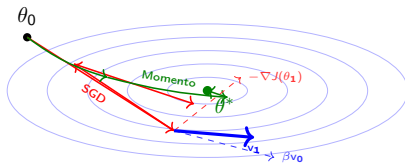


Figura: O vetor de momento (azul) acumula os gradientes (vermelho) e aprende a direção principal do vale.

A Ponte: De Pré-condicionamento a Momento V

Momento Acelerado de Nesterov (NAG)

O NAG refina a ideia com um passo “lookahead”. Em vez de calcular o gradiente na posição atual, ele primeiro dá um passo na direção do momento acumulado e calcula o gradiente *nessa posição futura*, usando-o para corrigir a trajetória.

$$\mathbf{v}_{t+1} \leftarrow \beta \mathbf{v}_t + \nabla J(\theta_t - \eta \beta \mathbf{v}_t)$$

$$\theta_{t+1} \leftarrow \theta_t - \eta \mathbf{v}_{t+1}$$

Otimizadores com Momento I

A Analogia da “Bola Pesada”

A ideia do momento é adicionar “inércia” à atualização dos pesos. O otimizador acumula uma média móvel dos gradientes passados, agindo como uma bola pesada que:

- **Acelera** em direções consistentes onde o gradiente aponta para o mesmo lado.
- **Amortece as oscilações** em direções onde o gradiente muda rapidamente.

Otimizadores com Momento II

A Regra de Atualização do SGD com Momento

Introduzimos uma variável de “velocidade” v e um hiperparâmetro de momento β :

$$v^{(t+1)} \leftarrow \beta v^{(t)} + \eta \nabla_{\theta} J(\theta^{(t)})$$

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - v^{(t+1)}$$

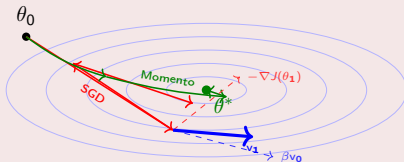


Figura: O SGD padrão (vermelho) oscila, enquanto o Momento (verde) segue um caminho mais suave e direto para o mínimo.

Otimizadores Adaptativos A: Adagrad I

A Ideia: Taxas de Aprendizado por Parâmetro

O Adagrad (Adaptive Gradient Algorithm) adapta a taxa de aprendizado para cada parâmetro θ_i individualmente, dando passos maiores para parâmetros com gradientes historicamente pequenos e passos menores para parâmetros com gradientes grandes.

Otimizadores Adaptativos A: Adagrad II

A Regra de Atualização

Ele consegue isso acumulando o quadrado dos gradientes passados em uma variável de estado s_t . Seja $g_t = \nabla J(\theta_t)$.

- 1 Acumule o gradiente ao quadrado:

$$s_t = s_{t-1} + g_t \odot g_t$$

- 2 Atualize o parâmetro:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t + \epsilon}} \odot g_t$$

Onde $\odot$ é o produto elemento a elemento e ϵ é uma constante pequena para evitar divisão por zero.

Otimizadores Adaptativos A: Adagrad III

O Ponto Fraco

O acumulador s_t só cresce. Com o tempo, o denominador se torna tão grande que a taxa de aprendizado efetiva vai a zero, parando prematuramente o treinamento.

Otimizadores Adaptativos B: RMSProp I

A Correção: Média Móvel Exponencial

O RMSProp (Root Mean Square Propagation) resolve o problema do Adagrad substituindo o acumulador por uma **média móvel exponencial** dos gradientes ao quadrado. Isso impede que o denominador cresça indefinidamente.

Otimizadores Adaptativos B: RMSProp II

A Regra de Atualização

Seja $\mathbf{g}_t = \nabla J(\theta_t)$ e β_2 um fator de decaimento (e.g., 0.999).

- 1 Atualize a média móvel do gradiente ao quadrado:

$$\mathbf{s}_t = \beta_2 \mathbf{s}_{t-1} + (1 - \beta_2)(\mathbf{g}_t \odot \mathbf{g}_t)$$

- 2 Atualize o parâmetro:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\mathbf{s}_t + \epsilon}} \odot \mathbf{g}_t$$

Otimizadores Adaptativos B: RMSProp III

A Ponte para o Adam

O RMSProp introduz a ideia de usar uma média móvel para adaptar as taxas de aprendizado, que é o segundo componente chave do Adam.

Juntando Tudo: Adam (Adaptive Moment Estimation) I

O Melhor dos Dois Mundos

O otimizador **Adam** combina a ideia de **momento** (uma média móvel dos gradientes, para acelerar) com a ideia de **taxas de aprendizado adaptativas** do RMSProp.

Diederik P. Kingma e Jimmy Ba (2015). “Adam: A Method for Stochastic Optimization”. Em: *ICLR 2015*

Sejam $\mathbf{g}_t = \nabla J(\theta_t)$, β_1 o decaimento do momento (e.g., 0.9) e β_2 o decaimento do segundo momento (e.g., 0.999).

- 1 Atualiza a média móvel do gradiente (1º momento):

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t$$

Juntando Tudo: Adam (Adaptive Moment Estimation) II

- 2 Atualiza a média móvel do gradiente ao quadrado (2º momento):

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2)(\mathbf{g}_t \odot \mathbf{g}_t)$$

- 3 (Opcional, mas importante) Corrige o viés das estimativas iniciais:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}$$

- 4 Atualização final do parâmetro:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$$

O Adam é o otimizador padrão na maioria das aplicações de Deep Learning devido à sua robustez e eficiência.

Questão de Pesquisa Recente: Mínimos Planos e Generalização I

Nem todos os Mínimos são Iguais

A pesquisa moderna sugere que a **geometria** do mínimo encontrado é mais importante do que seu valor de custo exato.

- **Mínimos “Afogados” (Sharp Minima):** Pequenas perturbações nos pesos (devido à diferença entre os dados de treino e teste) levam a um grande aumento no custo. Estes mínimos tendem a generalizar mal.
- **Mínimos “Planos” (Flat Minima):** A função de custo é relativamente insensível a pequenas mudanças nos pesos. Estes mínimos tendem a generalizar melhor, pois a solução é mais robusta.

Questão de Pesquisa Recente: Mínimos Planos e Generalização II

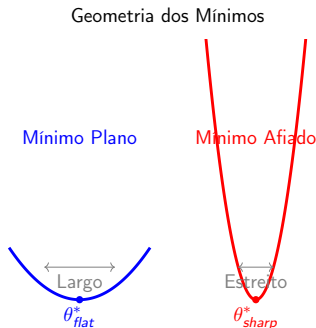


Figura: Um mínimo plano (largo) é mais robusto a perturbações do que um mínimo afiado (estreito).

Como o SGD Encontra Mínimos Planos? I

Formalizando a “Planicidade”

A “nitidez” (sharpness) de um mínimo pode ser medida pelo maior autovalor (λ_{max}) da sua matriz Hessiana $\nabla^2 J(\theta)$. Mínimos planos têm uma Hessiana com autovalores pequenos. [Sepp Hochreiter e Jürgen Schmidhuber \(1997\)](#). “Flat minima”. Em: *Neural computation* 9.1, pp. 1–42

Como o SGD Encontra Mínimos Planos? II

A Hipótese do Ruído do SGD

O ruído inerente à estimativa do gradiente em mini-batches age como um **regularizador implícito**. A distribuição estacionária dos parâmetros θ tende a se concentrar em regiões onde a função de custo $J(\theta)$ é baixa e a “difusão” (relacionada ao ruído do gradiente) é alta.

Pode-se mostrar que o SGD tem maior probabilidade de convergir para mínimos onde a Hessiana é pequena em relação ao ruído do gradiente. Nitish Shirish Keskar et al. (2017). “On large-batch training for deep learning: Generalization gap and sharp minima”. Em: *International Conference on Learning Representations (ICLR)*

Como o SGD Encontra Mínimos Planos? III

Intuição

Imagine uma bolinha quicando em uma superfície. É muito mais provável que a bolinha “escape” de um poço estreito e afiado do que de uma bacia larga e plana. O ruído do SGD age de forma análoga, empurrando o otimizador para fora de mínimos afiados e instáveis.

O Efeito Rashomon: Múltiplos Modelos Bons I

Definição (Leo Breiman)

O Efeito Rashomon descreve o fenômeno, comum em problemas complexos, onde existe um grande conjunto de modelos com performance preditiva virtualmente indistinguível (o "Rashomon set").

O Efeito Rashomon: Múltiplos Modelos Bons II

Conexão com Mínimos Planos

A existência de **mínimos planos** fornece a intuição geométrica para o Efeito Rashomon.

Uma bacia larga e plana na paisagem de custo é uma região onde **muitos conjuntos de parâmetros θ diferentes** (ou seja, muitos modelos distintos) podem “viver”, todos alcançando um erro de treino e de teste similarmente baixo. A robustez do mínimo plano permite a existência de um grande Rashomon set.

O Efeito Rashomon: Múltiplos Modelos Bons III

A “Mágica”: Liberdade de Escolha

Se múltiplos modelos são igualmente bons em predição, não estamos mais presos a encontrar o “único” modelo melhor. Em vez disso, podemos procurar dentro do Rashomon set por um modelo que satisfaça outras propriedades desejáveis, sem sacrificar a acurácia:

- **Simplicidade e Interpretabilidade**
- **Justiça (Fairness)** e equidade
- **Monotonicidade** ou outras restrições de domínio

O Efeito Rashomon: Múltiplos Modelos Bons IV

Leitura Adicional

Para uma visão aprofundada sobre a inevitabilidade e as implicações do Efeito Rashomon:

Cynthia Rudin et al. (2024). “Position: amazing things come from having many good models”. Em: *ICML*

Lucas Monteiro Paes et al. (2023). “On the Inevitability of the Rashomon Effect”. Em: *IEEE Journal on Selected Areas in Information Theory* 4.2, pp. 416–427. DOI: 10.1109/JSAIT.2023.3287661

Revisitando o Trade-off Viés-Variância Clássico I

A Decomposição Clássica do Erro

No cenário clássico (modelos lineares, poucos parâmetros), o erro de predição esperado em um ponto x para um modelo h treinado em um dataset $\mathcal{D}$ pode ser decomposto em:

$$\mathbb{E}_{\mathcal{D}}[(y - h(x; \mathcal{D}))^2] = \underbrace{(\mathbb{E}_{\mathcal{D}}[h(x; \mathcal{D})] - f(x))^2}_{(\text{Viés})^2} + \underbrace{\mathbb{E}_{\mathcal{D}}[(h(x; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[h(x; \mathcal{D})])^2]}_{\text{Variância}} + \sigma_{\text{ruído}}^2$$

Revisitando o Trade-off Viés-Variância Clássico II

A Curva em U

A intuição clássica, baseada neste trade-off, diz que à medida que a complexidade do modelo aumenta:

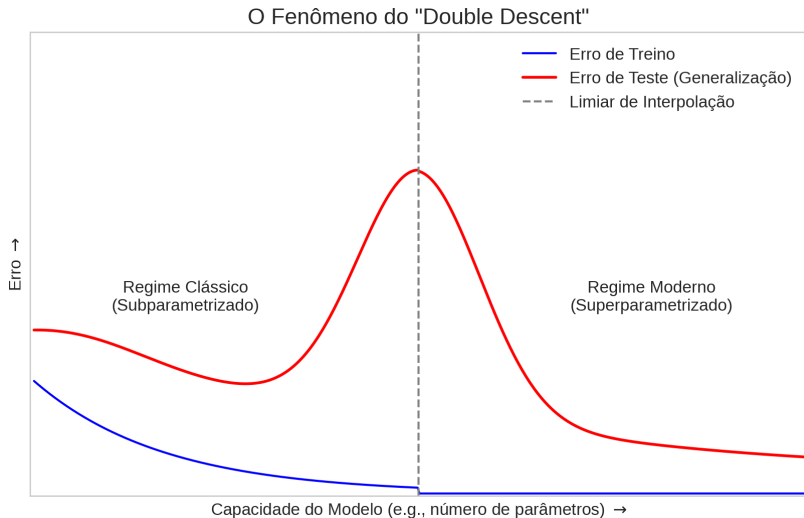
- O **viés** diminui (o modelo se ajusta melhor).
- A **variância** aumenta (o modelo fica mais sensível aos dados de treino).
- O erro de teste total segue uma curva em U, com um ponto ótimo de complexidade.

O Fenômeno Moderno do "Double Descent" I

Desafiando a Curva Clássica

Em redes neurais profundas e outros modelos superparametrizados, observa-se um fenômeno diferente. [Mikhail Belkin et al. \(2019\)](#). “Reconciling modern machine-learning practice and the classical bias–variance trade-off”. *Em: Proceedings of the National Academy of Sciences* 116.32, pp. 15849–15854

O Fenômeno Moderno do "Double Descent" II



Mergulho Raso: O Viés Implícito do SGD I

O que acontece no Regime Superparametrizado?

Quando o erro de treino é zero, existem *infinitas* soluções (conjuntos de pesos θ) que interpolam perfeitamente os dados. Qual delas o otimizador escolhe?

Mergulho Raso: O Viés Implícito do SGD II

O Viés Implícito da Descida de Gradiente

A descida de gradiente, ao ser inicializada perto da origem, tem um **viés implícito**: ela tende a encontrar, entre todas as soluções de erro zero, aquela com a **menor norma Euclidiana** $||\theta||_2$.

A decomposição do risco neste regime muda. O erro pode ser aproximado por: viés da solução de norma mínima somada a variância da norma mínima. Conforme aumentamos a superparametrização, a norma da solução mínima tende a diminuir, o que por sua vez reduz a variância, explicando a segunda descida da curva. [Trevor Hastie et al. \(2022\)](#). “Surprises in high-dimensional ridgeless least squares interpolation”. Em: *The Annals of Statistics* 50.2, pp. 949–986

Encerramento e Perguntas I

Resumo da Aula

- Analisamos por que a Descida de Gradiente é a base da otimização em larga escala.
- Contrastamos o mundo ideal da otimização convexa com a realidade não-convexa das redes neurais.
- Exploramos otimizadores avançados como Momento e Adam, que aceleram e estabilizam o treinamento.
- Discutimos tópicos de pesquisa modernos, como a importância dos mínimos planos e o fenômeno do double descent.

Encerramento e Perguntas II

Para a Próxima Aula

- **Tópico:** Laboratório de Otimizadores e Regularização.
- **Objetivo:** Implementar e comparar visualmente o desempenho do SGD, Momento e Adam, e investigar o efeito de diferentes técnicas de regularização.

Perguntas?

Referências I

-  Bach, Francis (2025). *Learning Theory from First Principles*.
URL: https://www.di.ens.fr/~fbach/ltpf_book.pdf.
-  Belkin, Mikhail et al. (2019). “Reconciling modern machine-learning practice and the classical bias–variance trade-off”. Em: *Proceedings of the National Academy of Sciences* 116.32, pp. 15849–15854.
-  Bishop, Christopher M. (2006). *Pattern Recognition and Machine Learning*. Springer.
-  Bishop, Christopher M. e Hugh Bishop (2023). *Deep Learning: Foundations and Concepts*. Springer.
-  Buchanan, Sam et al. (ago. de 2025). *Learning Deep Representations of Data Distributions*.
<https://ma-lab-berkeley.github.io/deep-representation-learning-book/>. Online.

Referências II

-  Cybenko, George (1989). “Approximation by superpositions of a sigmoidal function”. Em: *Mathematics of control, signals and systems* 2.4, pp. 303–314.
-  Domke, Justin (2011). *Automatic Differentiation and Neural Networks*. Lecture Notes, Statistical Machine Learning. URL: https://people.cs.umass.edu/~domke/courses/sml2011/08autodiff_nnets.pdf.
-  Gefter, Amanda (fev. de 2015). “The Man Who Tried to Redeem the World with Logic”. Em: *Nautilus*. URL: <https://nautil.us/the-man-who-tried-to-redeem-the-world-with-logic-235253/>.
-  Grünwald, Peter D. (2007). *The Minimum Description Length Principle*. The MIT Press.

Referências III

-  Hastie, Trevor et al. (2022). “Surprises in high-dimensional ridgeless least squares interpolation”. Em: *The Annals of Statistics* 50.2, pp. 949–986.
-  Hochreiter, Sepp e Jürgen Schmidhuber (1997). “Flat minima”. Em: *Neural computation* 9.1, pp. 1–42.
-  Jordan, Michael I (2019). “Artificial intelligence—the revolution hasn’t happened yet”. Em: *Harvard Data Science Review* 1.1.
-  Keskar, Nitish Shirish et al. (2017). “On large-batch training for deep learning: Generalization gap and sharp minima”. Em: *International Conference on Learning Representations (ICLR)*.
-  Kingma, Diederik P. e Jimmy Ba (2015). “Adam: A Method for Stochastic Optimization”. Em: *ICLR 2015*.
-  Krause, Andreas e Jonas Hübner (2025). *Probabilistic Artificial Intelligence*. arXiv: 2502.05244 [cs.AI].

Referências IV

-  Liu, Yuxi (jan. de 2024). *The Perceptron Controversy*. Website. URL: <https://yuxi.ml/essays/posts/perceptron-controversy/> (acesso em 10/09/2025).
-  Martins, André FT e Ramón F Astudillo (2016). “From softmax to sparsemax: A sparse model of attention and multi-label classification”. Em: *International conference on machine learning*. PMLR, pp. 1614–1623.
-  McCulloch, Warren S. e Walter Pitts (1943). “A Logical Calculus of the Ideas Immanent in Nervous Activity”. Em: *The Bulletin of Mathematical Biophysics* 5.4, pp. 115–133. DOI: [10.1007/bf02478259](https://doi.org/10.1007/bf02478259).

Referências V

-  Mercer, James (1909). “Functions of positive and negative type, and their connection with the theory of integral equations”. *Em: Philosophical transactions of the Royal Society of London. Series A* 209.441-458, pp. 415–446.
-  Minsky, Marvin e Seymour A Papert (1969). *Perceptrons: An introduction to computational geometry*. MIT press.
-  Montufar, Guido F et al. (2014). “On the number of linear regions of deep neural networks”. *Em: Advances in neural information processing systems* 27.
-  Murphy, Kevin Patrick (2023). *Probabilistic Machine Learning: Advanced Topics*. The MIT Press.
-  Nesterov, Yurii (2004). *Introductory lectures on convex optimization: A basic course*. Vol. 87. Springer Science & Business Media.

Referências VI

-  Oja, Erkki (1982). “A simplified neuron model as a principal component analyzer”. Em: *Journal of mathematical biology* 15.3, pp. 267–273.
-  Paes, Lucas Monteiro et al. (2023). “On the Inevitability of the Rashomon Effect”. Em: *IEEE Journal on Selected Areas in Information Theory* 4.2, pp. 416–427. DOI: 10.1109/JSAIT.2023.3287661.
-  Prince, Simon J.D. (2023). *Understanding Deep Learning*. The MIT Press.
-  Rasmussen, Carl Edward e Christopher K. I. Williams (2006). *Gaussian processes for machine learning*. The MIT Press.
-  Rosenblatt, Frank (1958). “The perceptron: a probabilistic model for information storage and organization in the brain.”. Em: *Psychological review* 65.6, p. 386.

Referências VII

-  Rudin, Cynthia et al. (2024). “Position: amazing things come from having many good models”. Em: *ICML*.
-  Schölkopf, Bernhard e Alexander J Smola (2002). *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press.
-  Shalev-Shwartz, Shai e Shai Ben-David (2014). *Understanding machine learning: From theory to algorithms*. Cambridge university press.
-  Shannon, C. E. (1988). “Programming a computer for playing chess”. Em: *Computer Chess Compendium*. Berlin, Heidelberg: Springer-Verlag, pp. 2–13. ISBN: 0387913319.
-  Shannon, Claude E. (1948). *A Mathematical Theory of Communication*. Vol. 27, pp. 379–423, 623–656.

Referências VIII

-  Turing, Alan M. (1969). "Intelligent Machinery". Em: *Machine Intelligence 5*. Ed. por Bernard Meltzer e Donald Michie. Edinburgh University Press, pp. 3–23.
-  Vapnik, Vladimir (1998). *Statistical learning theory*. Wiley.
-  Vieira, Tim (ago. de 2017). *Backprop is not just the chain rule*. Blog Post. URL: <https://timvieira.github.io/blog/post/2017/08/18/backprop-is-not-just-the-chain-rule/> (acesso em 15/09/2025).